

# Session 10 Teamwork — Close the Same Gap on Workspaces

## Context

---

Work in your team (six teams of four students). The instructor's own `CreateProjectDto` just had its three `@Transform(trim)` callbacks type-guarded — a non-string `name / key / description` now returns a clean `400` instead of the wrong status code. `CreateWorkspaceDto` has the identical, unguarded `.trim()` calls — your job is to close it at the source, the same way CORE just did for Projects, and then verify the full error picture across both resources.

## Goal

---

`CreateWorkspaceDto`'s own `name / slug / description` reach the same end state as Projects' own DTO — reasoned through by your team, not copied and renamed from `projects/dto/*.ts`. Then, as a team, confirm every kind of error response in the app behaves the way today's class says it should.

## What to build

---

- `CreateWorkspaceDto` (in `src/workspaces/dto/create-workspace.dto.ts`): add the same type guard, `typeof value === 'string' ? value.trim() : value`, inside each of the three existing `@Transform(({ value }) => value.trim())` callbacks — `name`, `slug`, and `description`.
- `UpdateWorkspaceDto` needs no changes at all — it already extends `PartialType(CreateWorkspaceDto)`, so the guarded callback reaches `PATCH` automatically, the same way it did for Projects.
- A completed error-behavior matrix: as a team, trigger a validation failure, a missing resource, and a duplicate resource against BOTH `/projects` and `/workspaces`, and write down, for each, which part of the app produced the response (a DTO's own decorators, the service's own error, or the filter).

## Rules

---

**Allowed:** everything the instructor just used for Projects — the type guard pattern, exactly as shown in class (no new decorators, no new packages, no changes to the global filter). **Not yet:** no `@UseFilters` on a specific controller (that was FLEX discussion only), no new response fields, no guards, no database.

## Test it

---

BASH

```
curl -i -X POST localhost:3000/workspaces -H "Content-Type: application/json" -d '{"name": 123, "slug": "tw-test"}'
curl -i -X PATCH localhost:3000/workspaces/1 -H "Content-Type: application/json" -d '{"description": 42}'
curl -i -X POST localhost:3000/workspaces -H "Content-Type: application/json" -d '{"name": "Acme ", "slug": "acme-tw"}'
curl -i -X GET localhost:3000/projects/999
curl -i -X GET localhost:3000/workspaces/999
```

## Done when

---

- `POST /workspaces` with `name / slug / description` non-string returns a clean `400`, not the global filter's generic `500`.
- `PATCH /workspaces/:id` with a non-string value behaves the same way, with zero changes to `update-workspace.dto.ts` itself.
- A valid, padded `name` is still correctly trimmed and stored.
- Your team has a completed matrix and every member can explain why `UpdateWorkspaceDto` needed no changes at all, and which mechanism produces each row of the matrix.

## Working time

---

About 30 minutes — roughly 15 for the DTO fix (a smaller, focused repeat of a pattern you already watched built once) and 15 for the matrix verification.